

Android Framework 16 심화

Day 4 — APEX · RRO · SystemUI · WindowManager · Android Automotive

이미지를 다시 굽지 않고 시스템을 바꾸는 법 → Car SystemUI 커스터마이징 → CarService · CarPropertyManager · VHAL → 버튼 한 번을 Perfetto 로 4일 전체 잇기



오늘의 일정 — 8시간

Part	내용	소요
Part 0 환경 실습	실습 1 writable-system · 2단계 remount · aapt · 실습 디렉토리	15분
Part 1 강의	Ch 1 시스템을 바꾸는 세 가지(APEX·RRO/SRO·소스) · Ch 2 SystemUI·WindowManager · Ch 3 Automotive	90분
Part 2 실습	실습 2~13 (APEX → RRO → ListOverlays → dumpsys → Car SystemUI RRO → QS 타일 → Perfetto → WMS 앱 → Automotive 계층 → HVAC 연동 → VHAL 조작 → 버튼 한 번 flow)	335분
Part 3 정리	4일 총정리 · Worksheet	40분



오늘의 축: 실습 2·3·7 = 이미지 없이 바꾸는 세 가지 · 실습 6·7 = 같은 SystemUI 를 리소스/코드로 · 실습 10~13 = Day 2 규칙이 Automotive 에 그대로

Part 0 · 실습 1 — writable-system · 2단계 remount · aapt

1

Emulator -writable-system 부팅

bat 파일에 -writable-system 확인

2

2단계 remount

adb root → remount(verity 해제) → reboot → root → remount → remount succeeded

3

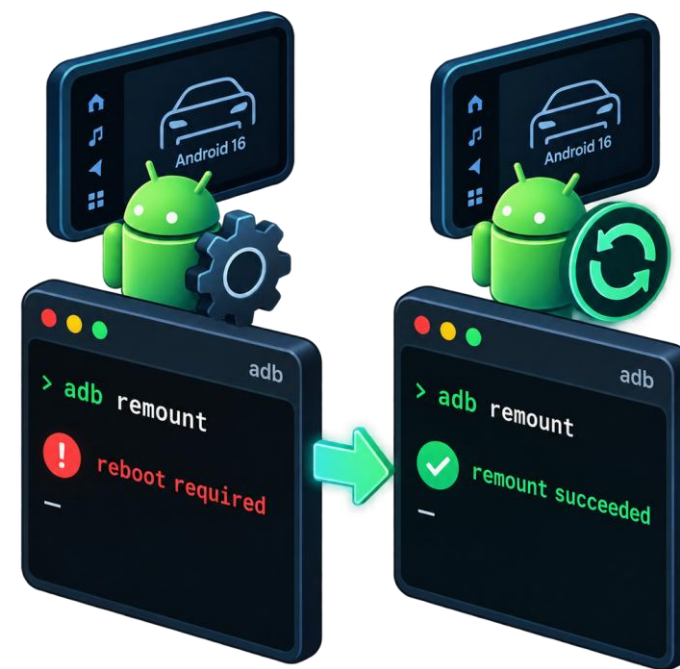
aapt PATH

%LOCALAPPDATA%\Android\Sdk\build-tools\35.0.0 (실습 6)

4

디렉토리 · 프로젝트

simple_apex(키 4종 확인) · RRO_test/overlay_list · zip 3종 Gradle sync



소요 15분 · remount succeeded 가 실습 2·6·7·11 의 전제

Chapter 1

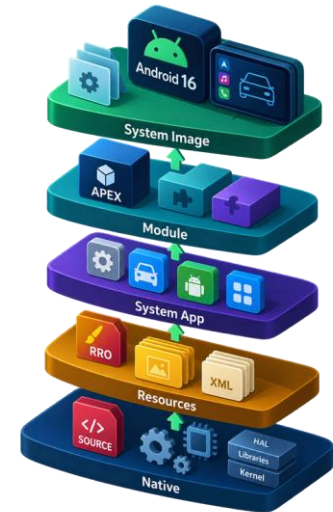
시스템을 바꾸는 세 가지 방법 — APEX · RRO / SRO · 소스 수정

30분 · 델타 배포 판단표 · APEX 구조 · RRO 계약과 매칭 · Car SystemUI
특수성 · SRO · 변경이 안 먹을 때 체크리스트



델타 배포 판단표 — "이미지 다시 구워야 하나요?"

바꾸는 것	필요한 절차	시간	오늘
Native 바이너리 · .so	m <모듈> + adb push /data	수십 초	Day 2·3
Java 리소스 (색·문자열·drawable·config)	RRO — APK 설치 + cmd overlay enable	분	실습 3·6
시스템 앱 코드 (SystemUI)	m SystemUI + APK push(remount) + killall	수 분	실습 7
독립 배포 모듈	APEX — .apex push + reboot	수 분	실습 2
init.rc · sepolicy	init_rc 는 push, .te 는 정책 리빌드	경계	Day 2 실습 9
커널 · 기본 탑재 RRO · VHAL 속성	m emu_img_zip + Emulator 재시작	수십 분	Day 1 실습 13



▶ 애니메이션 1 · 델타 배포 판단표

이미지를 다시 굽는 것은 마지막 줄뿐이다. 나머지는 push · install · enable 로 끝난다.

APEX — 모듈을 파일시스템 이미지로 묶다

```
apex {
    name: "com.example.simple",
    manifest: "apex_manifest.json",           // name · version
    binaries: ["simple_apex_binary"],          // cc_binary { apex_available: [.
    ..] }
    key: "com.example.simple.key",             // apex_key: .pem + .avbpubkey
    (AVB)
    certificate: ":com.example.simple.certificate", // .pk8 + .x509.pem (A
    PK)
    updatable: false,                          // 이미지 내장 - compressible 불가
}
// file_contexts 는 명시하지 않는다 → system/sepolicy/apex/com.example.simple-f
file_contexts
```

m com.example.simple

\$OUT/system/apex/*.apex

push /system/apex + re
boot

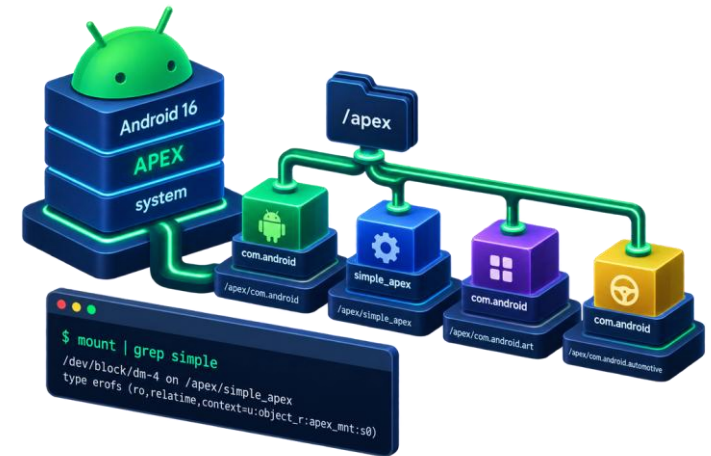
remount 상태

apexd 검증 · loop moun
t

/apex/<name>@<ver>

/apex/.../bin/ 실행

dumpsys apexservice



- 서명 두 벌: AVB(부팅 검증) + APK(패키지)
- file_contexts 위치가 틀리면 should be under system/sepolicy
- updatable:false + compressible:true → 빌드 오류
- Mainline(ART·Media·Wi-Fi)이 이 형식으로 Play 업데이트

▶ 애니메이션 2 · APEX

RRO — 세 당사자의 계약: overlayable · overlay · OverlayManager

타겟 앱 — overlayable.xml

```
<overlayable name="RROTargetTheme">
  <policy type="public">
    <item type="string" name="hello_world"/>
  </policy>
  <item type="drawable" name="ic_target_icon"/>
</overlayable>
```

오버레이 APK — <overlay> (hasCode=false)

```
<overlay
  android:targetPackage="com.example.rrotarget"
  android:targetName="RROTargetTheme"
  android:priority="1"
  android:isStatic="false"/>
```

cmd overlay list	의미	다음
[x]	활성	확인
[]	매칭 OK · 비활성	enable
---	매칭 실패 (NO_IDMAP)	targetName / overlayable 확인



- 같은 이름·타입만 덮는다 — 파일명이 아니라 리소스 이름
- priority 높은 활성 오버레이가 이긴다 (실습 4 충돌 실험)
- SRO(Product Flavors)는 같은 매칭이 빌드 시 Gradle 에서
- IOverlayManager hidden API = cmd overlay 의 실제 (ListOverlays)

▶ 애니메이션 3 · RRO 매칭

Car SystemUI 특수성 — overlayable 미션언 → /system/app 설치



mTargetOverlayableName: null

사용자=public(계약 필수)
/system/app=system(불필요)

targetName 제거 + /system/ap
p

Car 전용 이름 추출

```
adb root; adb remount      # 1차: verity 해제 → reboot required
adb reboot
adb root; adb remount      # 2차: remount succeeded ← 실습 1 에서 완료
adb shell mkdir -p /system/app/SystemUIOverlay
adb push systemuioverlay.apk /system/app/SystemUIOverlay/
adb reboot
cmd overlay enable com.example.systemuioverlay; killall com.android.systemui
```



변경이 안 먹을 때 7가지

① remount succeeded? ② --- (targetName)? ③ enable 안 함? ④ killall? ⑤ aapt 이름? ⑥ 높은 priority 가 되나? ⑦ AndroidRuntime 크래시?

바 배경은 투명 리소스(*_transparent) 를 함께 덮어야 바뀐다 — Phone SystemUI 와 이름·구조가 다르다

▶ [애니메이션 4 · Car SystemUI RRO](#)

APEX 만들기 — simple_apex

목표 바이너리 하나를 APEX 로 묶어 빌드·설치하고 /apex 마운트 확인, file_contexts.compressible 오류 재현 · 소요 35분

진행 순서

1. binary/main.cpp + binary/Android.bp (apex_available)
2. apex_manifest.json · Android.bp (apex · apex_key · android_app_certificate)
3. 키 4종 확인 · system/sepolicy/apex/com.example.simple-file_contexts
4. m com.example.simple → deapexer list → WinSCP → push /system/apex → reboot
5. ls /apex/com.example.simple/ → 실행 → dumsys apexservice · Step 7 오류 2종 재현

확인 포인트

- \$OUT/system/apex/com.example.simple.apex
- /apex/com.example.simple@1 마운트
- Hello from Simple APEX 출력
- should be under system/sepolicy · do not compress 메시지



▶ 애니메이션 2 · APEX

RRO 문자열·색·drawable → ListOverlays · priority 충돌 · SRO

목표 overlayable 계약으로 5가지 리소스를 런타임에 바꾸고(실습 3), hidden API 도구·우선순위·Flavor 를 확인(실습 4) · 소요 35 + 20분

진행 순서

1. 실습 3: RROTarget (overlayable.xml 5개 항목, minSdk 34) → RROOverlay (<overlay> · hasCode=false · 같은 이름 리소스)
2. 실습 3: adb install ×2 → cmd overlay list [] → enable → force-stop-start → dump p → disable · 계약 실험
3. 실습 4: ListOverlays.java + Android.bp(java_binary, libs framework) → mm → jar WinSCP → app_process /system/bin ListOverlays
4. 실습 4: RROOverlay2 priority=2 → 둘 다 enable → 어느 것이 이기나 → dump priority
5. 실습 4: productFlavors original/themed → themedDebug Run (5분)

확인 포인트

- 5가지 리소스 변경 · overlayable 에서 뺀 것은 그대로
- ListOverlays 출력에 STATE_NO_IDMAP(---) 항목
- priority 2 문자열이 이김
- themedDebug 초록 배경



▶ 애니메이션 3 · RRO 매칭

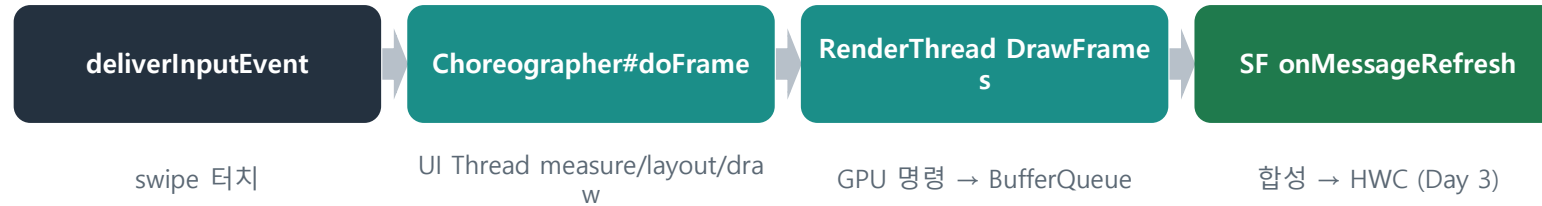
Chapter 2

SystemUI · WindowManager — 구조 · 분석 · 커스터마이징

25분 · dumpsys 분석 · QS 타일 (QSTileImpl · Dagger) · 프레임 파이프라인 · 창 타입과 권한 · WMS 창 → SF Layer



SystemUI — 프레임 파이프라인과 QS 타일 추가



```

public class CpuInfoTile extends QSTileImpl<BooleanState> {
    public static final String TILE_SPEC = "cpuinfo";
    @Inject public CpuInfoTile(QSHost host, QsEventLogger l, @Background Looper bg
    , @Main Handler h, ...) { super(...); }
    protected void handleClick(View v) { mShowing = !mShowing; refreshState(); }
    protected void handleUpdateState(BooleanState s, Object arg) { s.label = "CPU
    Info"; s.icon = ...; s.secondaryLabel = readLoadAvg(); }
}
// Dagger 등록 - 없으면 SystemUI 크래시
@Binds @IntoMap @StringKey(CpuInfoTile.TILE_SPEC) QSTileImpl<?> bindCpuInfoTile(Cp
uInfoTile t);
// config.xml: quick_settings_tiles_default = "wifi,bt,...,cpuinfo" → m SystemUI
→ push → killall
  
```

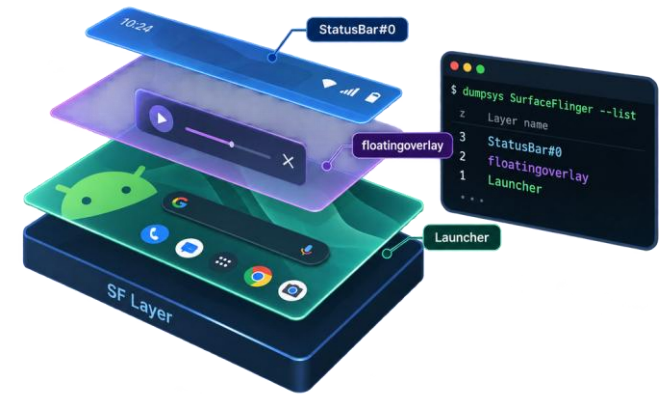


- Java App 은 Thread 2개(UI + Render) — Jank 원인을 두 트래킹으로
- `dumpsys gfxinfo Janky %` = Perfetto Actual Timeline 의 숫자 판
- "타일이 안 보여요" → Dagger 누락(`logcat -s AndroidRuntime`) 또는 `Car config.xml`
- 상태바 커스텀 뷰: `status_bar.xml` + `TextView` 상속 (참고 코드)

▶ 애니메이션 5 · SystemUI 프레임 + QS 타일

WindowManager — 창 타입 · 권한 · WMS 창은 SF Layer

```
WindowManager.LayoutParams p = new WindowManager.LayoutParams(
    WRAP_CONTENT, WRAP_CONTENT,
    WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY,    // 2038 - 다른
    WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE, PixelFormat.TRANSLUCENT);
windowManager.addView(floatingView, p);    // Binder → WMS.addWindow → 권한 검사 → SF createSurface
// 권한: SYSTEM_ALERT_WINDOW - Settings.canDrawOverlays() / appops set <pkg>
// 없으면 BadTokenException: permission denied for window type 2038
```



창 타입	누가	예
TYPE_APPLICATION	Activity	앱 창
TYPE_APPLICATION_OVERLAY	권한 있는 앱	플로팅 위젯
시스템 창 (StatusBar 등)	system UID	SystemUI 의 바

- WMS 가 Day 3 실습 9 의 createSurface 를 대신 → App 은 Surface 를 받아 그린다
- 권한 검사는 WMS(system_server) 에서 — Day 2 UID 검사와 같은 자리
- dumpsys window windows ↔ dumpsys SurfaceFlinger --list 로 대응
- Fragment 분할은 Layer 1개 · 디스플레이가 다르면 Layer 스택이 다르다

▶ 애니메이션 6 · WMS 창 → SF Layer

SystemUI dumpsys 분석 → Car SystemUI RRO

목표 dumpsys 로 상태바·알림·프레임 통계 읽기(실습 5) → aapt 로 이름 뽑아 /system/app 에 오버레이 설치(실습 6) · 소요 15 + 30분

진행 순서

1. 실습 5: dumpsys statusbar 섹션 · 알림 발생(am start VIEW) 전후 notification · svc wifi 토글 · gfxinfo Janky
2. 실습 6: cmd overlay list --- · dumpsys overlay mTargetOverlayableName: null 확인
3. 실습 6: pm path → adb pull CarSystemUI.apk → aapt dump resources | findstr color/system_bar
4. 실습 6: SystemUIOverlay (targetName 없음, minSdk 34) → mkdir /system/app/SystemUIOverlay → push → reboot
5. 실습 6: cmd overlay list [] → enable → killall com.android.systemui → screencap

확인 포인트

- mDisabled · Tiles · Icons 위치, Janky %
- --- 와 null 확인
- aapt 로 실제 이름 6개 이상
- 시계 핑크 · 아이콘 노랑 · Driver 시안



▶ 애니메이션 4 · Car SystemUI RRO

QS 타일 소스 수정 → SystemUI Perfetto → WindowManager 앱

목표 코드로 SystemUI 바꾸기(실습 7) · UI/RenderThread Jank(실습 8) · 플로팅 창과 Layer(실습 9) · 소요 40 + 20 + 25분

진행 순서

1. 실습 7: CpuInfoTile.java · ic_qs_cpu_info.xml · Dagger @Binds @IntoMap (grep FlashlightTile.TILE_SPEC 로 위치) · config.xml
2. 실습 7: m SystemUI → WinSCP CarSystemUI.apk → push /system_ext/priv-app/CarSystemUI/ → killall · Step 6 등록 빼고 크래시 로그
3. 실습 8: day4_sysui.cfg (gfx/view/wm/am/input) → QS swipe 4회 캡처 → doFrame / DrawFrames / SF · gfxinfo framestats
4. 실습 9: FloatingOverlayDemo → appops allow → dumpsys window · SurfaceFlinger --list → deny → BadTokenException
5. 실습 9: SplitScreenDemo Run → --list 예 Layer 하나 · dumpsys display

확인 포인트

- QS 패널에 CPU Info · load 값
- 등록 누락 시 로그에서 cpuinfo 찾기
- UI Thread / RenderThread 두 트랙, Jank 프레임
- 오버레이 Layer 확인 · permission denied for window type 2038



▶ 애니메이션 5 · SystemUI 프레임 + QS

▶ 애니메이션 6 · WMS 창 → Layer

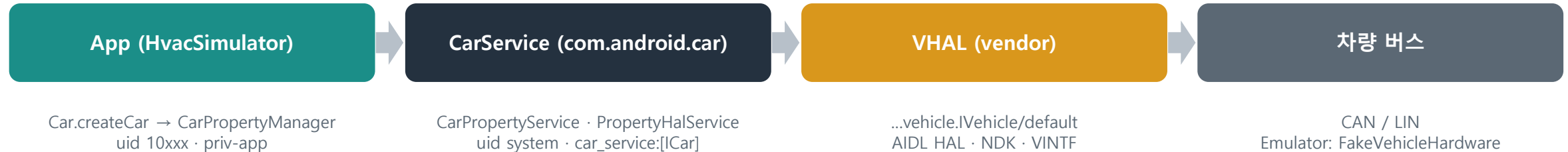
Chapter 3

Android Automotive — CarService · CarPropertyManager · VHAL

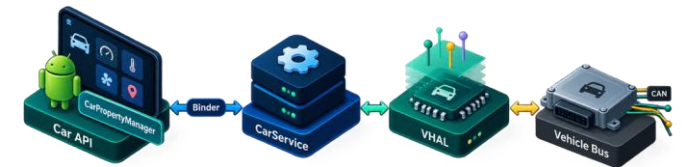
35분 · 계층 · Day 2 규칙 그대로 · Vehicle Property ID · CarPropertyManager API · VHAL 에뮬레이터 · cmd car_service · 권한 추적



Automotive 계층 — App → CarService → VHAL, Day 2 규칙 그대로



Day 2 규칙	Automotive 에서
Java System Service (실습 11)	CarService — service list grep car_service
AIDL HAL 이름 <pkg>.<l>/<inst> · vendor · NDK · VINTF (실습 10)	android.hardware.automotive.vehicle.IVehicle/default · /vendor/etc/vintf/manifest.xml
getCallingUid 권한 검사 (실습 8)	CarPropertyService — HVAC_* → CONTROL_CAR_CLIMATE
App 은 HAL 직접 못 부름	App 은 CarService 만 본다
Callback 은 Binder Thread (Day 3 실습 6)	onChangeEvent → runOnUiThread



```

dumsys ...IVehicle/default --set 0x15600503 -a 4
9 -f 25.0 # VHAL 값 변경 → 위로 전파
cmd car_service day-night-mode night / inject-v
hal-event / get-property-value

```

▶ 애니메이션 7 · Automotive 계층

Vehicle Property 와 CarPropertyManager API

```

car = Car.createCar(this, null, Car.CAR_WAIT_TIMEOUT_WAIT_FOREVER, (c, ready) -> {
    pm = (CarPropertyManager) c.getCarManager(Car.PROPERTY_SERVICE); // 준비된 뒤
   에만
});
CarPropertyConfig<?> cfg = pm.getCarPropertyConfig(VehiclePropertyIds.HVAC_TEMPERA
TURE_SET);
int[] areas = cfg.getAreaIds(); // Emulator: [49, 68] - 좌석
그룹
float t = pm.getProperty(Float.class, HVAC_TEMPERATURE_SET, areas[0]).getValue();
pm.setProperty(Float.class, HVAC_TEMPERATURE_SET, areas[0], 22.5f); // Binder →
CarService → VHAL
pm.subscribePropertyEvents(HVAC_TEMPERATURE_SET, callback); // onChange
Event: Binder Thread

```

HVAC_TEMPERATURE_SET = 0x15600503	
0x1_____	Group SYSTEM (vendor 는 0x2)
0x_5_____	Area SEAT (GLOBAL 0x1, WINDOW 0x3 ...)
0x_6_____	Type FLOAT (INT32 0x4, BOOLEAN 0x2)
0x__00503	속성 ID

권한

CAR_SPEED: dangerous → pm grant 가능
 CONTROL_CAR_CLIMATE: signature|privileged
 → /system/priv-app + privapp-permissions.xml
 거부 시 logcat -s CarPropertyService 에 필요
 한 권한 이름



- areald 는 1 이 아니라 config 가 주는 49/68 — 호출 전 getCarPropertyConfig
- 범위 밖 값, 잘못된 areald → IllegalArgumentException
- 값의 진실은 VHAL — inject 와 --set 의 차이
- 콜백 Thread 이름 Binder:... → runOnUiThread

▶ 애니메이션 7 · Automotive (Property · 권한)

Automotive 계층 확인 · cmd car_service · 권한 거부 추적

목표 세 프로세스와 Binder 서비스를 확인하고 Day 2 규칙과 대조, 차량 이벤트 주입, 권한 거부 → 권한 이름 찾기 · 소요 30분

진행 순서

1. `service list | grep -E "car_service|vehicle" · ps -A (uid) · dumphsys car_service --hal · --services CarPropertyService`
2. `dumphsys ...IVehicle/default --list / --get 0x15600503 -a 49 / -a 68`
3. `/vendor/etc/vintf/manifest.xml` 의 `automotive.vehicle` 항목 — Day 2 실습 10 표와 대조
4. `cmd car_service day-night-mode night → day · inject-vhal-event 0x11600207 60 · get-property-value · inject-key 3`
5. Kitchen Sink 권한 revoke → 속도 읽기 실패 → `logcat -s CarPropertyService` 에서 권한 이름 → `pm list permissions -g → grant`

확인 포인트

- car_service: [android.car.ICar] · IVehicle/default
- --get 값 · manifest 항목
- night 모드 즉시 전환 · 클러스터 속도 60
- 거부 로그의 `android.car.permission.CAR_SPEED`



▶ 애니메이션 7 · Automotive 계층

HvacSimulator → CarPropertyManager 연동 → VHAL 조작·실패 재현

목표 저장소의 가짜 HVAC UI 를 실제 속성에 연결하고 priv-app 으로 설치(실습 11), 밖에서 바꾸고 깨뜨려 보기(실습 12) · 소요 35 + 25분

진행 순서

1. 실습 11: `useLibrary("android.car") · uses-library · CONTROL_CAR_CLIMATE · Car.createCar → getCarPropertyConfig → arealds [49,68]`
2. 실습 11: 버튼 → `setProperty(HVAC_TEMPERATURE_SET / FAN_SPEED / POWER_ON) · subscribePropertyEvents → onChangeEvent → runOnUiThread`
3. 실습 11: `/system/priv-app/HvacSimulator/ + privapp-permissions-hvacsimulator.xml → reboot → granted=true`
4. 실습 12: VHAL `--set 0x15600503 -a 49 -f 25.0` → 앱 UI 25.0 · `inject-vhal-event` 와 `--get` 비교 · Kitchen Sink 동기화
5. 실습 12: 범위 밖 값 · `areald 0 · runOnUiThread` 제거 → 오류 3종 · xml 제거 → `SecurityException` → 권한 이름 → 복구

확인 포인트

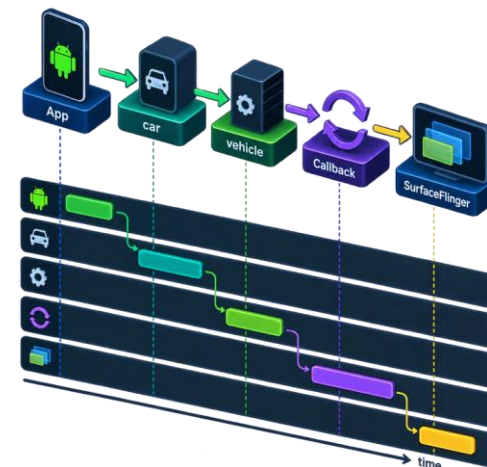
- `arealds=[49, 68]` 로그 · `onChangeEvent thread=Binder...`
- `dumpsys package granted=true`
- `--set` 후 앱 UI 변화 · subscriber 목록에 앱
- `IllegalArgumentException` 2종 · `CalledFromWrongThread` · `SecurityException`



▶ 애니메이션 7 · Automotive (Property · 권한)

실습 13 · Perfetto 로 온도 버튼 한 번 따라가기 — 4일의 지도

순서	트랙 · 슬라이스	배운 날
①	hvacsimulator main — deliverInputEvent → binder transaction (ICarProperty::setProperty)	Day 1 AIDL Proxy
②	com.android.car binder:N — binder reply (CarPropertyService)	Day 2 실습 11
③	같은 Thread — binder transaction IVehicle::setValues → VHAL binder reply	Day 2 실습 10
④	VHAL → CarService — onPropertyEvent (역방향)	Day 1 실습 9 · Day 2 실습 6
⑤	CarService → hvacsimulator binder:N — onChangeEvent (Looper 없음)	Day 3 실습 6
⑥	runOnUiThread → main Choreographer#doFrame → RenderThread DrawFrames	Day 3 실습 5 · Day 4 실습 8
⑦	surfaceflinger onMessageRefresh → HWC	Day 3 실습 9:10

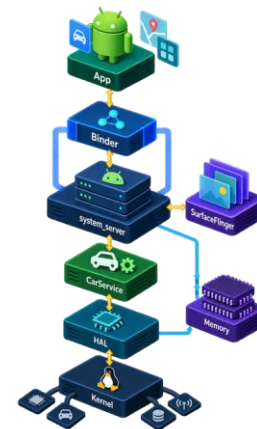


- `cfg = Day 3 cfg + aidl + atrace_apps hvacsimulator/com.android.car`
- 캡처 8초 동안 온도 ▲ 3번 → pull → 핀 4개
- Flow events 탭에서 6개 이상 화살표
- SQL 로 binder 쌍과 `dur_us` → "버튼 한 번의 비용" 기록

▶ [애니메이션 8 · 버튼 한 번의 flow](#)

4일 총정리 — "안 될 때 보는 순서" 와 델타 배포

순서	도구	무엇을 보나	배운 날
1	logcat -s AndroidRuntime	크래시 스택 (Dagger 누락 · 예외)	Day 4 실습 7
2	dumpsys <service>	서비스 상태 (statusbar · car_service · SurfaceFlinger · meminfo)	Day 2·3·4
3	cmd overlay list / service list	매칭 [x] [] --- · 서비스 등록 여부	Day 2 · Day 4
4	getenforce · logcat grep avc	SELinux 거부 (scontext · tcontext)	Day 2 실습 9
5	adb remount 상태	remount succeeded 인가	Day 4 실습 1
6	logcat -s CarPropertyService	권한 거부 → 필요한 권한 이름	Day 4 실습 10·12
7	Perfetto	어느 구간이 느린가 (flow · SQL)	Day 3·4 실습 13



현장 적용 제안

- ① 우리 제품 SystemUI 색·아이콘 RRO 1개
- ② 우리 HAL 속성 1개를 CarPropertyManager 로 읽는 앱
- ③ 서비스 하나를 Perfetto 로 캡처해 왕복 시간 기록

델타 배포 판단표(슬라이드 5) + 이 진단 순서 카드 — 4일간 쓴 도구를 문제 해결 순서로 재배열한 것

Day 4 Worksheet — 20문제, 20분

유형	문제	범위
객관식	Q1 ~ Q8	file_contexts 위치 · overlayable · --- · Car SystemUI · 2단계 remount · Dagger 누락 · SYSTEM_ALERT_WINDOW · Automotive 경로
단답형	Q9 ~ Q13	/apex/<name>@<ver> · hasCode · aapt dump resources · cmd car_service · IVehicle
빈칸 채우기	Q14 ~ Q17	AVB/APK 키 · targetPackage/targetName · doFrame/RenderThread · areald/getCarPropertyConfig
순번 지정	Q18 ~ Q19	Car SystemUI RRO 절차 · 온도 버튼 경로
선잇기	Q20	델타 배포 판단표

- Day4_Worksheet.html 을 열어 풀고 채점하기 → 문제별 해설 확인
- 종이로 풀 경우 인쇄 (문제만) → 강사가 정답·해설 모두 보기로 풀이
- 16 / 20 이상이면 오늘 내용을 충분히 이해한 것



▶ [Day 4 Worksheet 열기](#)

4일 과정 종료

Android Framework 16 심화 — 무엇을 가져가는가

Day 1

Binder 는 code + Parcel · Zygote fork · AIDL Stub/Proxy · DeathRecipient · Buffer
1 MB → FD

Day 2

libbinder Bn/Bp 를 손으로 · aidl_interface · getCallingUid · init.rc/SELinux · AIDL
HAL · VINTF

Day 3

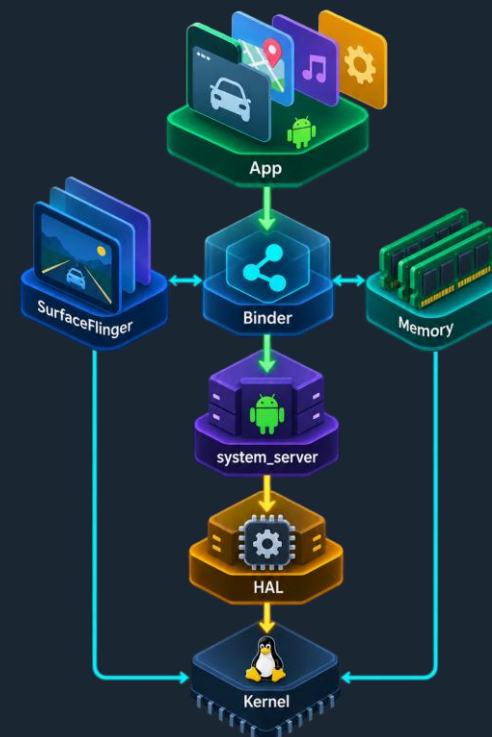
sp/RefBase · Thread · Looper 를 밑에서부터 · JNI · SurfaceFlinger BufferQueue · p
ipe/ashmem/memfd · LMKD

Day 4

델타 배포 판단표 · APEX · RRO 계약 · Car SystemUI · QS 타일 · WMS 창 · CarServ
ice → VHAL · 버튼 한 번의 flow

도구

adb shell · dumphsys · service/cmd · ftrace · strace · Perfetto — "안 될 때 보는 순
서" 로



수고하셨습니다. 실습 디렉토리(day2·day3·simple_apex)와 Emulator 이미지는 그대로 두고, 현장 적용 제안 3가지를 한 달 안에 해 보세요.